



兰州大学
LANZHOU UNIVERSITY

SCHOOL OF EARTH SCIENCES,
LANZHOU UNIVERSITY

第2章：计算机中数据表示



姚旭 (yaox@lzu.edu.cn)

王荣华，李亚军



课程安排

章节	周次	理论讲解	机房实践（与学科融合）
1	4-5	计算机简史与计算思维	命令行和图形界面、学科应用现状
2	6-7	计算机中数据表示	地质学科数据系统、大数据库
3	8-9	计算机系统组成	Office高级应用、微机硬软件配置
4	10-11	Python基本语法元素	Python编程开发环境配置
5	12-13	Python基本图形绘制	Python数据可视化实践
6	14-15	Python基本数据类型	Python数据和字符串基本操作
7	16-17	人工智能技术概述	Python基础语法回顾
8	18-19	人工智能方法及应用	人工智能的学科初步应用

姚旭、王荣华、李亚军



第二章：计算机中数据表示

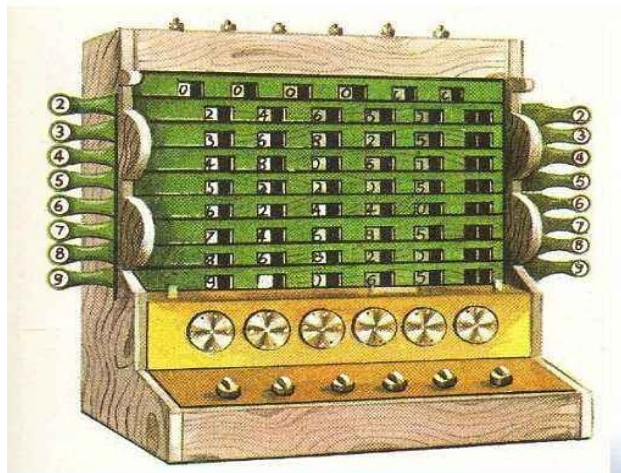
- 第一节：计算机中的数制
- 第二节：计算机中的数值
- 第三节：计算机中的字符
- 第四节：图像、音频和视频
- 第五节：条形码



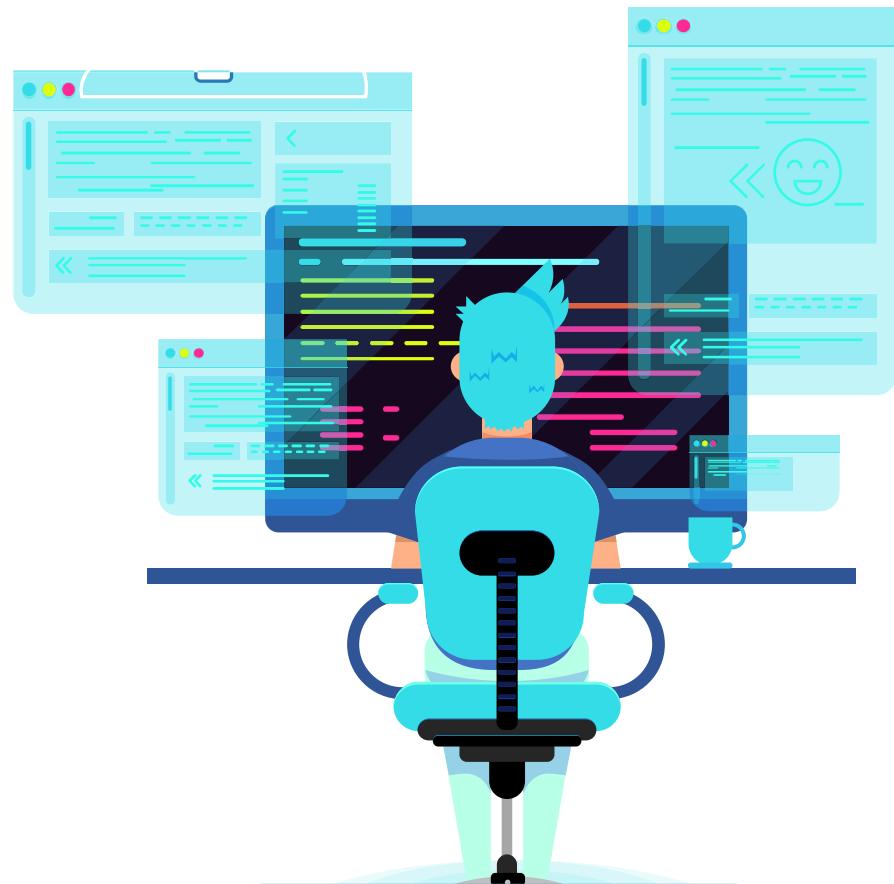
第一节：计算机中的数制



0-1二进制



数制的转换



二进制的运算



0和1

计算机中为什么使用二进制

- 最初计算机采用的电子元器件大都具有两种稳定的状态
- 二进制运算规则简单、运算速度快
- 两种状态分明，可靠性高
- “0”和“1”与逻辑量“假”和“真”相吻合，采用二进制便于实现逻辑运算

计算机中数据的单位

位

字节

字

位:

01 计算机中最小的数据单位, bit

02 n 个二进制位可以表示 2^n 种状态

计算机中数据的单位

位

字节

字

字节:

01 8位为一个字节, Byte

02 计算机中表示存储空间大小的基本单位

03 KB(2^{10} B), MB(2^{20} B), GB(2^{30} B), TB(2^{40} B)等等

计算机中数据的单位

位

字节

字

字:

01 计算机一次能够处理的二进制位的长度, word

02 字长64位, 即每个字由8个字节组成



数制转换

- 数制是由一组固定数码和一套统一的规则来表示数值的方法。
- 十进制数据使用10个数码（0、1、2、3、4、5、6、7、8、9）来表示，并按照“逢十进一”的规则进行运算。
- 二进制数使用2个数码（0、1）表示数据，而且运算规则是“逢二进一”。



基数

位权

按权
展开式

$$(N)_R = a_{n-1} \times R^{n-1} + a_{n-2} \times R^{n-2} + \dots \\ + a_2 \times R^2 + a_1 \times R^1 + a_0 \times R^0 + a_{-1} \times R^{-1} + \dots + a_{-m} \times R^{-m}$$

$$234.12 = 2 \times 10^2 + 3 \times 10^1 + 4 \times 10^0 + 1 \times 10^{-1} + 2 \times 10^{-2}$$



数制	数码	基数	位权	后缀
十进制	0, 1, 2, 3, 4, 5, 6, 7, 8, 9	10	10^i	D
二进制	0, 1	2	2^i	B
八进制	0, 1, 2, 3, 4, 5, 6, 7	8	8^i	Q
十六进制	0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F	16	16^i	H



非十进制数转换成十进制数：按权展开法

将二进制数1011.01B转换成十进制数。

$$1011.01B = 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 + 0 \times 2^{-1} + 1 \times 2^{-2} = 8 + 0 + 2 + 1 + 0 + 0.25 = 11.25D$$

将八进制数246Q转换成十进制数。

$$246Q = 2 \times 8^2 + 4 \times 8^1 + 6 \times 8^0 = 128 + 32 + 6 = 166D$$

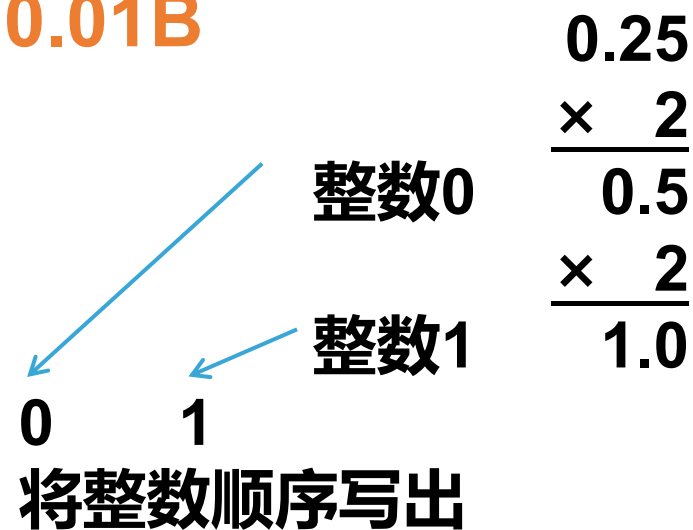
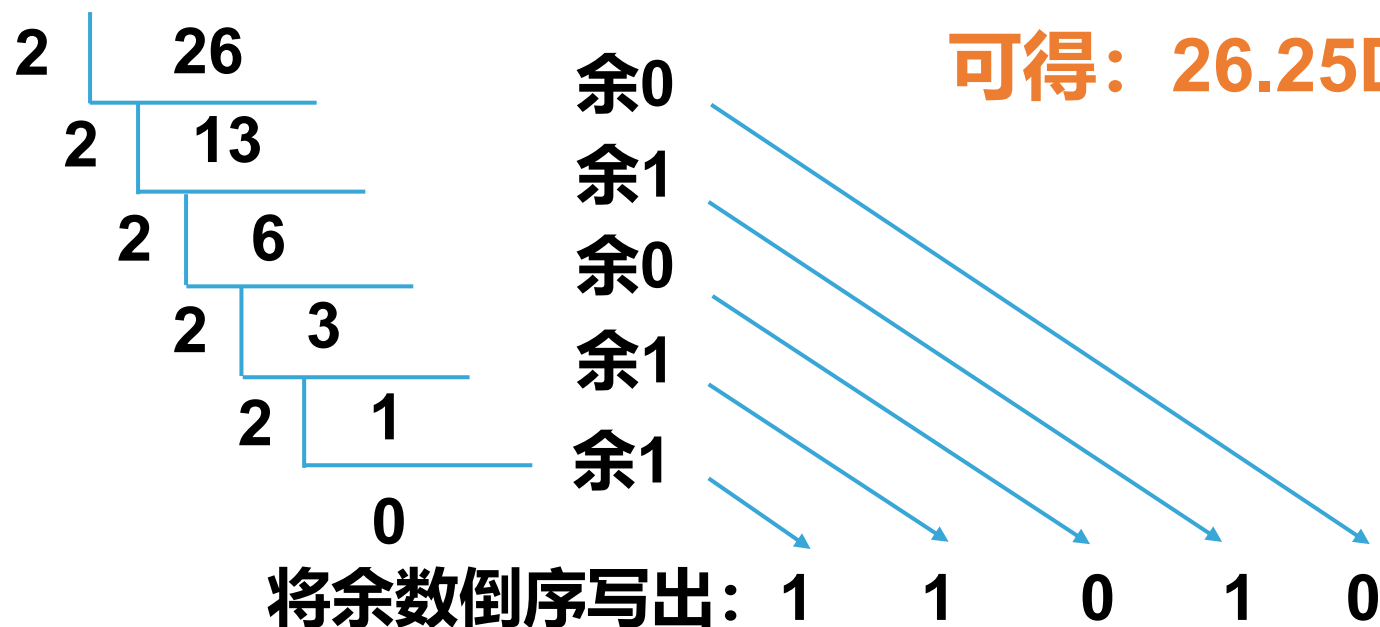
将十六进制数1A5.7H转换成十进制数。

$$1A5.7H = 1 \times 16^2 + 10 \times 16^1 + 5 \times 16^0 + 7 \times 16^{-1} = 256 + 160 + 5 + 0.4375 = 421.4375D$$



十进制数转换成非十进制数：除基取余法，乘基取整法

例2.4 将十进制数26.25D转换成二进制。



八进制呢？



二进制的算术 运算和逻辑运算





二进制数的算术运算

1011B+1101B的运算过程如下:

$$\begin{array}{r} 1011 \\ + 1101 \\ \hline 11000 \end{array}$$

11000B-1101B的运算过程如下?

$$\begin{array}{r} 11011 \\ - 1101 \\ \hline 1110 \end{array}$$



二进制数的逻辑运算





逻辑非运算规则

A	!A
1	0
0	1

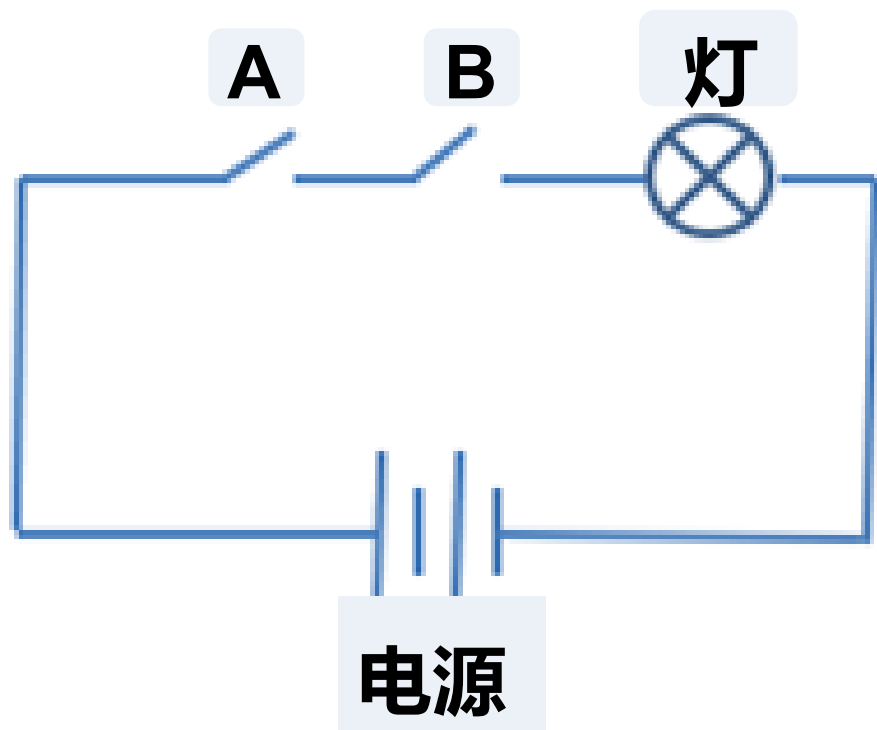


逻辑与运算规则

A	B	$A \wedge B$
0	0	0
0	1	0
1	0	0
1	1	1

通常使用 “ $\times$ ”或 “ $\wedge$ ” 表示逻辑与运算，当逻辑值A和B都为真时，它们逻辑与运算结果才为真，否则结果为假。

逻辑与运算规则



- 逻辑与运算类似于串联电路
- 串联电路中的两个开关都合上（真）时，电路才通，灯才亮（真）。

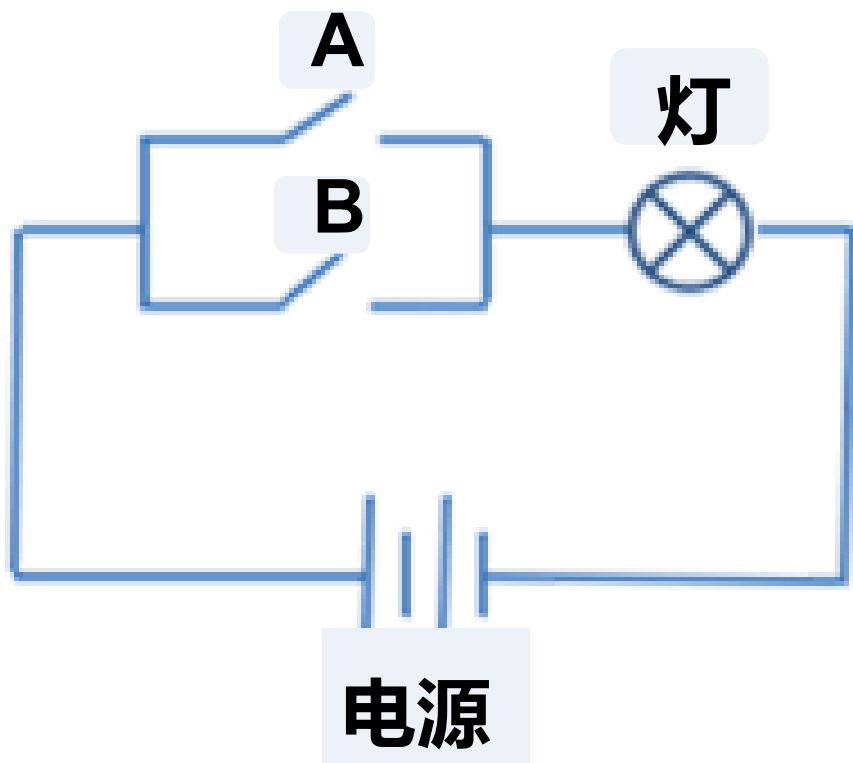


逻辑或运算规则

A	B	$A \vee B$
0	0	0
0	1	1
1	0	1
1	1	1

通常使用 “+”或 “ $\vee$ ” 表示逻辑或运算，当逻辑值A和B有一个为真时，它们逻辑或运算结果就为真，只有A和B都为假时，结果才为假。

逻辑或运算规则



- 逻辑或运算类似于并联电路
- 并联电路中的两个开关只要有一个合上（真）时，电路就能通，灯就会亮（真）。



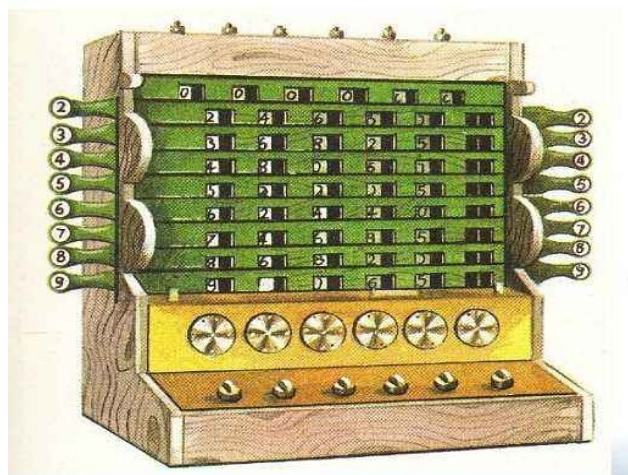
逻辑异或运算规则

A	B	$A \oplus B$
0	0	0
0	1	1
1	0	1
1	1	0

通常使用 “ $\oplus$ ” 表示逻辑异或运算，当逻辑值A和B相同时，它们逻辑异或运算结果为假；两个逻辑值相异时，结果才为真。



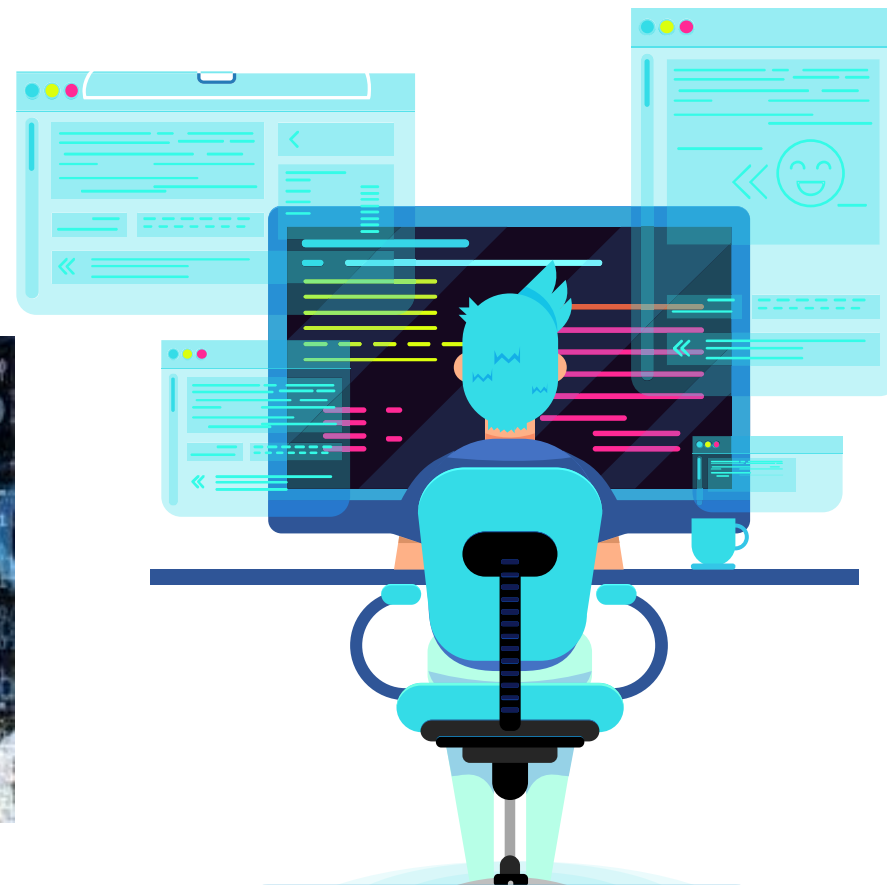
第二节：计算机中的数值



无、有符号数



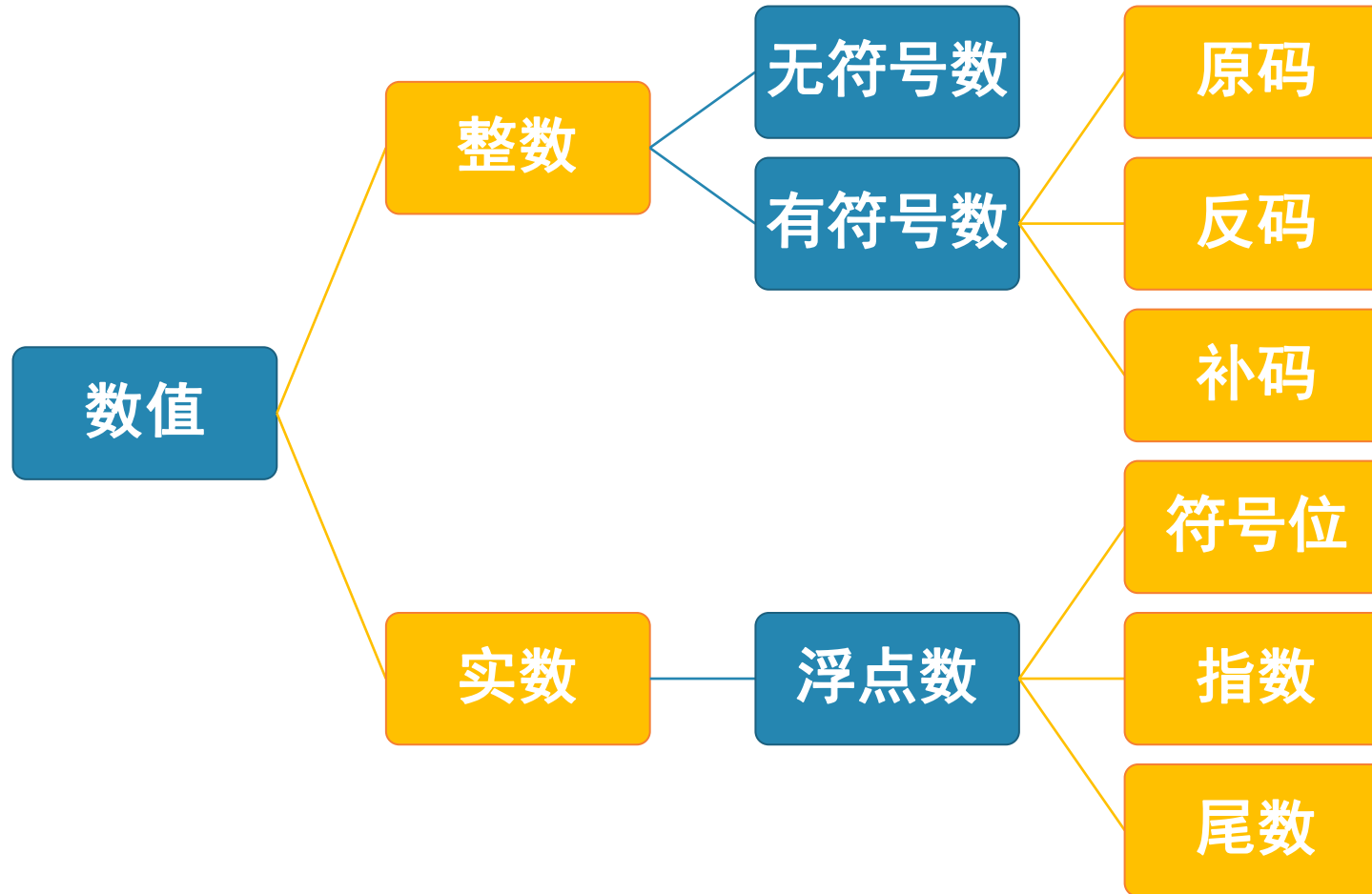
原-反-补码



实数



- 在计算机中，“数据”的表达范围非常广，不仅包括我们常说的数字、字符，还包括图形、图像、声音、动画、视频等各种媒体形式。
- 但不论哪种类型的数据，在计算机内部都是使用二进制表示的。
- 仅就**数值型数据**而言，包括**整型数据**和**实型数据**两种类型。





01 整型数据

整型 (Integer) 数据是不包含小数部分的数值型数据。

02 实型数据

实数，是有理数和无理数的总称。

实数经常用浮点数来表示。



无符号整数

你需要让整数有正负之分吗？

- 我们在使用数的时候，可以决定数的取值范围，也可以自己决定某个量是否需要区分正负。如果这个量出现负值是违反常识的，那么我们可以定义它为无符号整数（不带符号的整数）。
- 例如年龄、库存，就可以使用一个无符号整数来表示。当我们指定一个数量是无符号整数时，其最高位的1或0，和其它位一样，用来表示该数的大小。



- 以一个字节的长度（即8个比特位）为例，8位二进制数表示的最大无符号整数是“11111111”，即 $2^8-1=255$ ，表示的最小无符号整数是“00000000”，即0。
- 在计算机中，用于存储数据的位数越多，所能表示的数值的范围越大。



下表给出二进制的位数与所能表示的无符号整数的范围的关系。

位数	最大值	最小值
8 (1个字节)	$2^8-1=255$	0
16 (2个字节)	$2^{16}-1=65535$	0
32 (4个字节)	$2^{32}-1=4294967295$	0
64 (8个字节)	$2^{64}-1=18446744073709551615$	0



有符号整数



- 反之，当你需要表示的**数有正负之分的时候，就可以用二进制数中的最高位表示正负**。例如盈亏、坐标等等。那么最高位是哪一位呢？
- 如果最低位是第0位，那么，字长是单字节的数，最高位是第7位，字长是双字节的数，最高位是第15位，字长是四字节的数，最高位就是第31位……不同长度的数值类型，其最高位也就不同，但**总是最左边的那位**。



二进制的最高位（首位）表示符号

单字节数:

7	6	5	4	3	2	1	0
---	---	---	---	---	---	---	---



双字节数:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
----	----	----	----	----	----	---	---	---	---	---	---	---	---	---	---



- 最高位称为“**符号位**”：
- 为1时，表示该数为负值
- 为0时，表示为正值



有符号数的数据表示范围





以一个字节的数值对比

无符号数: 0 ----- 255

有符号数: -127 ----- 0 ----- 127

思考: 无符号数一共表示了256个数据 (0~255) ,
可是有符号数怎么才表示了255个数据呢 (-127~+127) ?



以一个字节的数值对比

最高位表示符号，将会出现两个0，一个正0（0000 0000），一个负0（1000 0000）。实际上表示的数字也是256个，只不过其中包含了两个0。

思考：请计算 $-1+1$ ？

$$\begin{aligned} &+1 \quad + \quad (-1) \\ &= 0000 \ 0001 + 1000 \ 0001 \\ &= 1000 \ 0010 \end{aligned}$$

按照有符号数的表示规则，1000 0010表示-2。为什么？



解决负数计算的问题——补码

01 原码

首位表示符号，其余表示数值

02 反码

除符号位外，其余数取反

03 补码

反码+1



解决负数计算的问题——补码

01 原码

首位表示符号，其余表示数值

$[+1]_{\text{原}} = 0000\ 0001$

$[-1]_{\text{原}} = 1000\ 0001$

例如，对于8位有符号整数，用最高位的二进制位表示符号：正数该位取0，负数该位取1。其余7位是数值位，存储数的绝对值，如果绝对值不足7位，则在左侧用0补齐。原码就是符号位加上真值的绝对值，即用第一位表示符号，其余位表示值。



解决负数计算的问题——补码

02 反码

除符号位外，其余数取反

$$[+1] = [00000001]_{\text{原}} = [00000001]_{\text{反}}$$

$$[-1] = [10000001]_{\text{原}} = [11111110]_{\text{反}}$$

反码是为了计算补码的一个中间过渡状态，计算公式是：正数的反码是其本身；负数的反码是在其原码的基础上，符号位不变，其余各个位取反。



解决负数计算的问题——补码

03 补码

反码+1

$$[+1] = [00000001]_{\text{原}} = [00000001]_{\text{反}} = [00000001]_{\text{补}}$$

$$[-1] = [10000001]_{\text{原}} = [11111110]_{\text{反}} = [11111111]_{\text{补}}$$

补码的计算公式是：正数的补码就是其本身；负数的补码是在其反码的基础上+1。



解决负数计算的问题——补码

01 原码

$[+1]_{\text{原}} = 0000\ 0001$

$[-1]_{\text{原}} = 1000\ 0001$

02 反码

$[+1] = [00000001]_{\text{原}} = [00000001]_{\text{反}}$

$[-1] = [10000001]_{\text{原}} = [11111110]_{\text{反}}$

03 补码

$[+1] = [00000001]_{\text{原}} = [00000001]_{\text{反}} = [00000001]_{\text{补}}$

$[-1] = [10000001]_{\text{原}} = [11111110]_{\text{反}} = [11111111]_{\text{补}}$



解决负数计算的问题——补码

写出+1、-1的8位补码形式，并用补码计算+1+ (-1)

$$\begin{aligned} & +1 \quad + \quad (-1) \\ & = 0000 \ 0001 + 1111 \ 1111 \\ & = 0000 \ 0000 \end{aligned}$$

由于字长为 8 位，最高位产生的进位被丢弃，所以8位数为0。

补码还有一些特殊的性质:

01

$[x+y]_{\text{补}} = [x]_{\text{补}} + [y]_{\text{补}}$,
即两数之和的补码等于各自补码的和。

02

$[x-y]_{\text{补}} = [x]_{\text{补}} + [-y]_{\text{补}}$,
即两数之差的补码等于被减数的补码
与减数相反数的补码之和。

03

$[[x]_{\text{补}}]_{\text{补}} = [x]_{\text{原}}$, 即按求补的方法,
对 $[x]_{\text{补}}$ 再求补一次, 结果等于 $[x]_{\text{原}}$ 。

04

硬性规定 $[-128]_{\text{补}} = 10000000$



实型数据

- 所有实数的集合则可称为实数系 (Real Number System) 或实数连续系统。理论上, 任何实数都可以用无限小数的方式表示, 小数点的右边是一个无穷的数列 (可以是循环的, 也可以是非循环的)
- 在计算机领域, 由于计算机只能存储有限的小数位, **实数经常用浮点数**来表示。



浮点数表示

十进制: $26.25 = 10^2 \times (0.2625)$

二进制: $11010.01 = 2^4 \times (1.101001)$

浮点数	占用内存	内存分配	指数范围	指数偏移	范围 (约等于)	精度
float	32位	1符号位 8指数位 23尾数位	[-127, 128]	+127	$[-3.4 \times 10^{38}, -1.18 \times 10^{-38}]$ U $[1.18 \times 10^{-38}, 3.4 \times 10^{38}]$	约7位 有效 数
double	64位	1符号位 11指数位 52尾数位	[-1023, 1024]	+1023	$[-1.8 \times 10^{308}, -2.23 \times 10^{-308}]$ U $[2.23 \times 10^{-308}, 1.8 \times 10^{308}]$	约16 位有 效数

IEEE754标准提供了如何在计算机内存中, 以二进制的方式存储十进制浮点数的具体标准。

标准是IEEE754-2008, 32位浮点数类型叫做 float, 64位浮点数叫做 double



浮点数表示

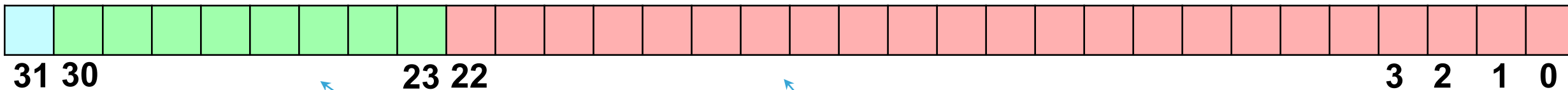
十进制: $26.25 = 10^2 \times (0.2625)$

二进制: $11010.01 = 2^4 \times (1.101001)$

1位
符号
号

(11位)
8位指数

(52位)
23位尾数



数值的正负

指数部分: 尾数中的
小数点应当向左
或向右移动的位数

尾数部分:
表示数值的有效数字

浮点数表示

十进制: $26.25 = 10^2 \times (0.2625)$

二进制: $11010.01 = 2^4 \times (1.101001)$

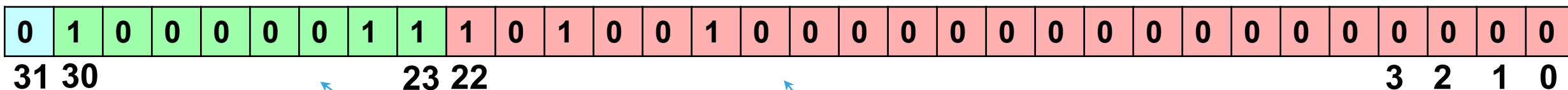
1位 符号

(11位)

8位指数

(52位)

23位尾数



数值的正负

指数部分：尾数中的 小数点应当向左 或向右移动的位数

4+127=131
1000 0011

尾数部分： 表示数值的有效数字

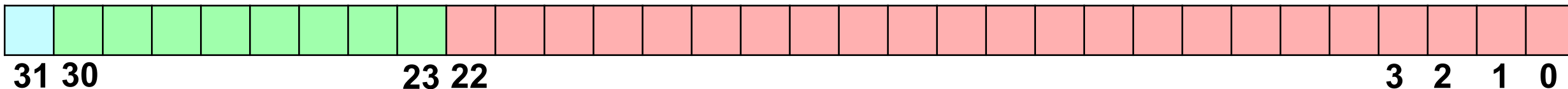
23位尾数，小数点之后尾数 并补零
1010 0100 0000 0000 000

浮点数表示

0 如何表示：全为0

26.25在64位中如何表示

0 10000011 101001000000000000000000



64位, 1符号位+11指数位+52尾数位, 指数范围[-1023, 1024], 指数偏移+1023

[illegible]



浮点数表示

十进制: $26.25 = 10^2 \times (0.2625)$

二进制: $11010.01 = 2^4 \times (1.101001)$

需要对
尾数特
殊处理

➤ 隐藏高位1.

尾数部分的最高位始终为1。比如1.101001，规范化之后，第一位只能为1。存储尾数时省略前面的1和小数点，只记录尾数中小数点之后的部分，这样就节约了一位内存。所以这里只需记录剩余的尾数部分: 101001

尾数指的其实是隐藏了整数部分1. 之后, 剩下的小数部分

➤ 2. 低位补0

多数时候尾数会不够填满尾数位(即图中的红色格子)，需要在低位补零，补齐23位。补零之后是: 1010 0100 0000 0000 000 (补至23位)

➤ 用二进制数表示十进制浮点数时，表示为尾数*指数的形式，并把尾数的小数点放在第一位和第二位之间，然后保证第一位数非0，这个处理过程叫做规范化(normalized)



第三节：计算机中的字符

0/1世界中 英文字符和中文字符



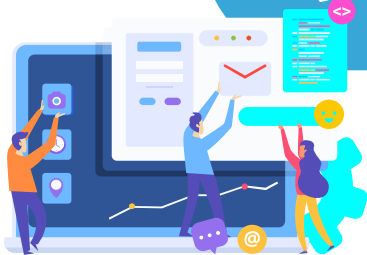


英文字符编码



英文字符普遍采用
ASCII(American
Standard Code for
Information
Interchange)码，即美国
信息交换标准代码。

英文字符编码



- ASCII码有7位版本和8位版本两种，国际上通用的是7位版本，7位版本的ASCII码有128个元素，其中控制字符34个，阿拉伯数字10个，大小写英文字母52个，各种标点符号和运算符号32个。
- 在计算机中实际用8位表示一个字符，最高位为“0”



二进制	十进制	字符/缩写	解释
00000000	0	NUL (NULL)	空字符
00000001	1	SOH (Start Of Heading)	标题开始
00000010	2	STX (Start Of Text)	正文开始
00000011	3	ETX (End Of Text)	正文结束
00000100	4	EOT (End Of Transmission)	传输结束
00000101	5	ENQ (Enquiry)	请求
00000110	6	ACK (Acknowledge)	回应/响应/收到通知
00000111	7	BEL (Bell)	响铃
00001000	8	BS (Backspace)	退格
00001001	9	HT (Horizontal Tab)	水平制表符
00001010	10	LF/NL (Line Feed/New Line)	换行键
00001011	11	VT (Vertical Tab)	垂直制表符
00001100	12	FF/NP (Form Feed/New Page)	换页键
00001101	13	CR (Carriage Return)	回车键
00001110	14	SO (Shift Out)	不用切换
00001111	15	SI (Shift In)	启用切换
00010000	16	DLE (Data Link Escape)	数据链路转义
00010001	17	DC1/XON (Device Control 1/Transmission On)	设备控制1/传输开始
00010010	18	DC2 (Device Control 2)	设备控制2
00010011	19	DC3/XOFF (Device Control 3/Transmission Off)	设备控制3/传输中断
00010100	20	DC4 (Device Control 4)	设备控制4
00010101	21	NAK (Negative Acknowledge)	无响应/非正常响应/ 拒绝接收
00010110	22	SYN (Synchronous Idle)	同步空闲
00010111	23	ETB (End of Transmission Block)	传输块结束/终止
00011000	24	CAN (Cancel)	取消
00011001	25	EM (End of Medium)	已到介质末端/介质存 储已满/介质中断
00011010	26	SUB (Substitute)	替补/替换
00011011	27	ESC (Escape)	逃离/取消
00011100	28	FS (File Separator)	文件分割符
00011101	29	GS (Group Separator)	组分隔符/分组符
00011110	30	RS (Record Separator)	记录分离符
00011111	31	US (Unit Separator)	单元分隔符

00100000	32	(Space)
00100001	33	!
00100010	34	"
00100011	35	#
00100100	36	\$
00100101	37	%
00100110	38	&
00100111	39	'
00101000	40	(
00101001	41	)
00101010	42	*
00101011	43	+
00101100	44	,
00101101	45	-
00101110	46	.
00101111	47	/
00110000	48	0
00110001	49	1
00110010	50	2
00110011	51	3
00110100	52	4
00110101	53	5
00110110	54	6
00110111	55	7
00111000	56	8
00111001	57	9
00111010	58	:
00111011	59	;
00111100	60	<
00111101	61	=
00111110	62	>
00111111	63	?
01000000	64	@

01000001	65	A
01000010	66	B
01000011	67	C
01000100	68	D
01000101	69	E
01000110	70	F
01000111	71	G
01001000	72	H
01001001	73	I
01001010	74	J
01001011	75	K
01001100	76	L
01001101	77	M
01001110	78	N
01001111	79	O
01010000	80	P
01010001	81	Q
01010010	82	R
01010011	83	S
01010100	84	T
01010101	85	U
01010110	86	V
01010111	87	W
01011000	88	X
01011001	89	Y
01011010	90	Z
01011011	91	[
01011100	92	\
01011101	93	]
01011110	94	^
01011111	95	_
01100000	96	`
01100001	97	a

01100010	98	b
01100011	99	c
01100100	100	d
01100101	101	e
01100110	102	f
01100111	103	g
01101000	104	h
01101001	105	i
01101010	106	j
01101011	107	k
01101100	108	l
01101101	109	m
01101110	110	n
01101111	111	o
01110000	112	p
01110001	113	q
01110010	114	r
01110011	115	s
01110100	116	t
01110101	117	u
01110110	118	v
01110111	119	w
01111000	120	x
01111001	121	y
01111010	122	z
01111011	123	{
01111100	124	
01111101	125	}
01111110	126	~
01111111	127	DEL (Delete)

表中列出了全部128个符号的ASCII码
0——48
A——65
a——97



中文字符编码



区位码
国际码
机内码

所有的国标汉字与符号组成一个 94×94 的矩阵，在此方阵中，每一行称为一个“区”（区号为01~94），每一列称为一个“位”（位号为01~94），该方阵实际组成了一个94个区，每个区内有94个位的汉字字符集，每一个汉字或符号在码表中都有一个唯一的位置编码，叫该字符的区位码。



区位码
国际码
机内码

区位码(十进制) → 区位码 (十六进制) + 2020H = 国标码
国标码 (十六进制) + 8080H = 机内码



汉字“中”的编码在汉字矩阵中排54行48列

区位码
国际码
机内码

01

$$(54\ 48)\ D = (36\ 30)\ H$$

区位码

02

$$(36\ 30)\ H + (20\ 20)\ H = (56\ 50)\ H$$

国标码

03

$$(56\ 50)\ H + (80\ 80)\ H = (D6\ D0)\ H$$

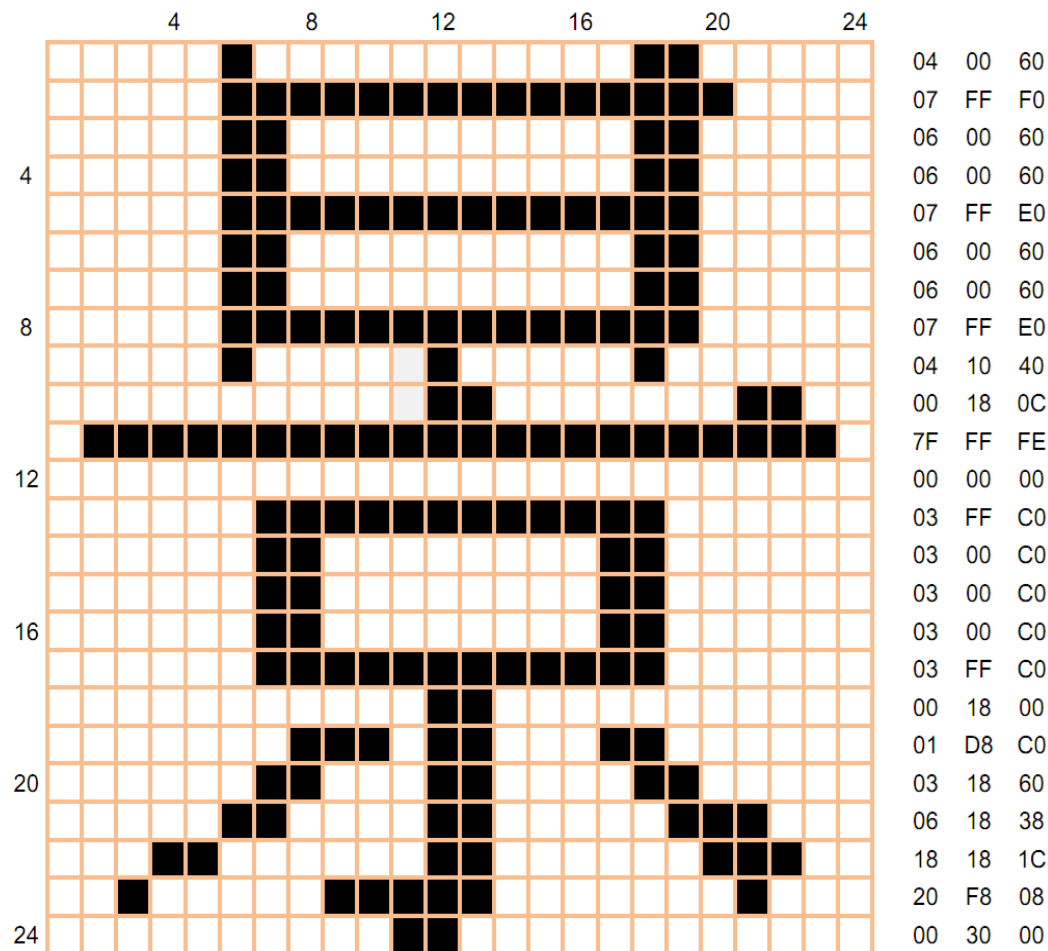
机内码



字形码

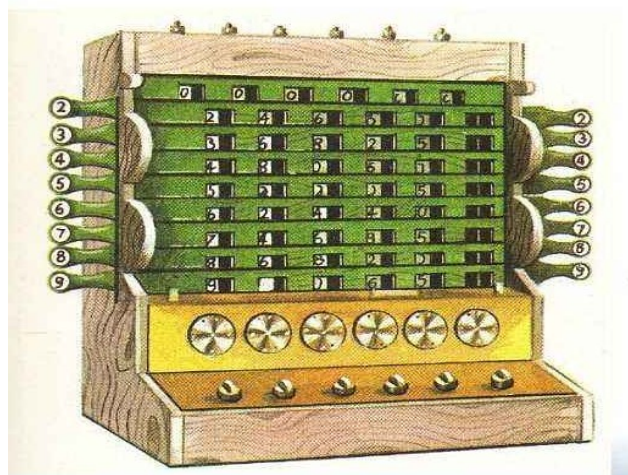
一个汉字的字形码就好像是一个黑白图像（二值图像）的存储码一样，仅用黑白两种颜色来表示图画。

24×24点阵的字形码需要用72字节($24 \times 24 \div 8 = 72$)表示。





第四节：图像音频和视频



图像



音频



视频



0/1世界中的图片、声音和视频

随着计算机技术、通信和网络技术、信息处理技术和人机交互技术的发展，拓展了信息的表示和传播方式，形成了将文字、图形图像、声音、动画和超文本等各种媒体进行综合、交互处理的多媒体技术。



位图

位图图像的数字化

黑白两色图像用1位位图表示 ($2^1=2$)

16色图像用4位位图表示 ($2^4=16$)

256色图像用8位位图表示 ($2^8=256$)

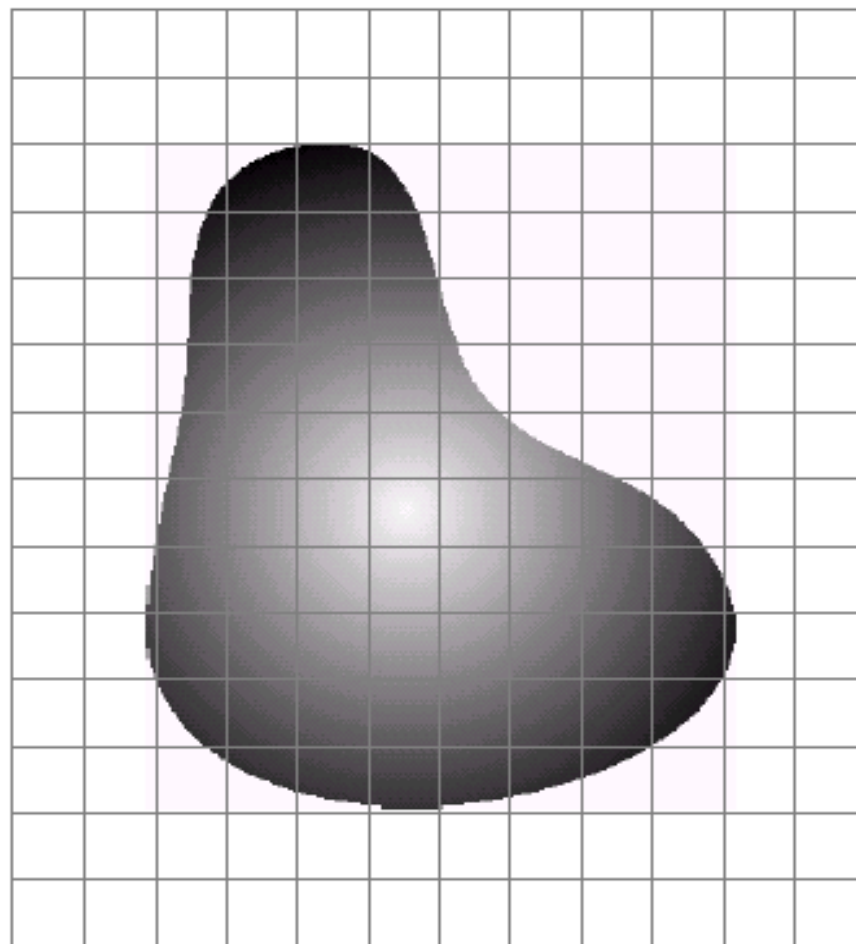
65536色图像用16位位图表示 ($2^{16}=65536$)

24位则可以表示1670多万种颜色 ($2^{24}=16777216$)

采样(Sampling)

采样就是要用多少点来描述一幅图像，也就是图像分辨率。例如：一幅640*480分辨率的图像，表示这幅图像是由 $640*480=307200$ 个像素点组成。显然，划分的网格越细密，图像分辨率就越高。

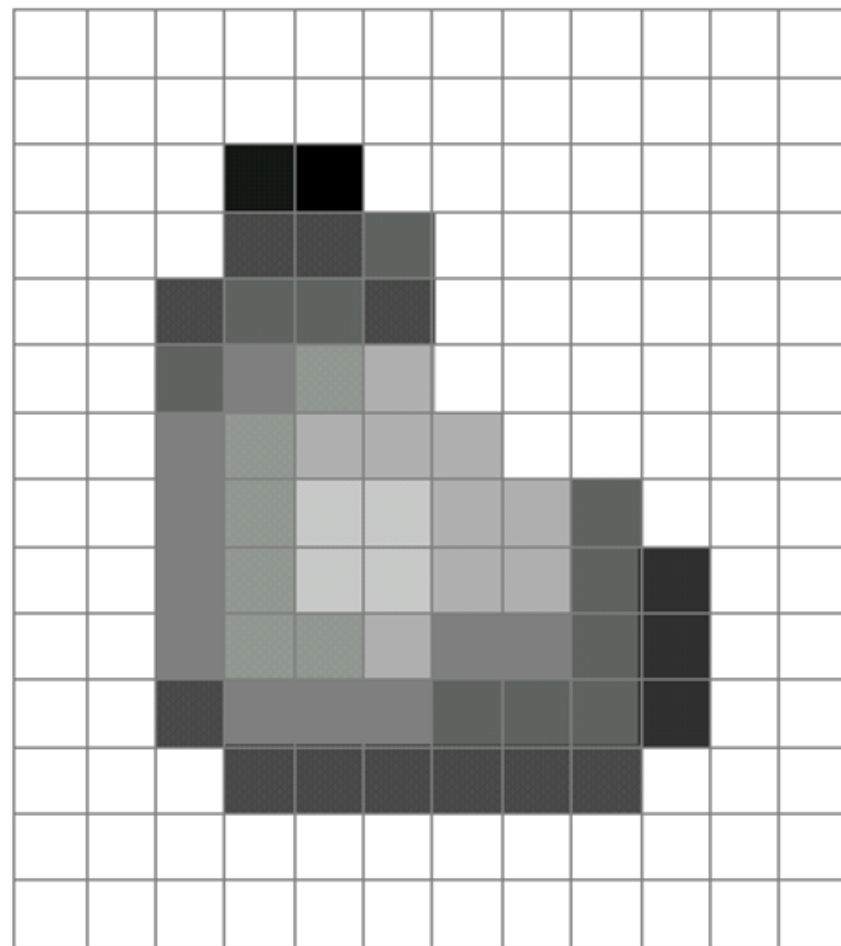
位图图像的数字化



量化(Quantization)

量化是指要使用多大范围的数值来表示图像采样之后的每一个点。量化的结果是图像能够容纳的颜色总数，它反映了采样的质量。

位图图像的数字化



压缩编码

将量化的图像用数字表示，如图是一个黑白灰度图像，分辨率为14*12，我们用1个字节来表示每个像素点的颜色，那么共有 $2^8=256$ 种深浅不同的灰色，最深的灰色是黑（0），最浅的灰色是白（255）。

位图图像的数字化

255	255	255	255	255	255	255	255	255	255	255	255
255	255	255	255	255	255	255	255	255	255	255	255
255	255	255	0	0	255	255	255	255	255	255	255
255	255	255	13	13	68	255	255	255	255	255	255
255	255	27	65	65	8	255	255	255	255	255	255
255	255	68	...	...	...	255	255	255	255	255	255
255	255	136	...	...	...	...	255	255	255	255	255
255	255	136	...	...	...	...	...	...	255	255	255
255	255	136	...	...	...	...	...	...	0	255	255
255	255	136	...	...	...	...	...	...	0	255	255
255	255	13	...	...	...	...	...	...	0	255	255
255	255	255	13	13	13	13	13	13	255	255	255
255	255	255	255	255	255	255	255	255	255	255	255
255	255	255	255	255	255	255	255	255	255	255	255



矢量图

矢量图是一种描述性的图像，**它存储的是量化后的数字**。矢量图的元素有直线、矩形、椭圆形、多边形、正文等。矢量图多用于计算机辅助设计（CAD）、三维动画等一些电脑创作。



视频信号是一帧帧连续播放的图像

- 在时间维上把视频分为离散的一帧一帧的图像，在时间维度上进行了抽样。
- 在时间方向和垂直方向上的抽样间距是由模拟电视系统的制式所决定的。
- 在每一帧图像内以垂直方向将图像离散为一条一条的扫描行，在空间维度上进行了抽样。



视频信号是一帧帧连续播放的图像

01

NTSC制式

02

PAL制式

03

**SECAM制
式**



视频信号的计算

显示分辨率设为1024*768的图像，颜色为24位真彩色，采用NTSC制式制作1分钟视频文件，在不考虑压缩的情况下，它占用存储空间有多大？

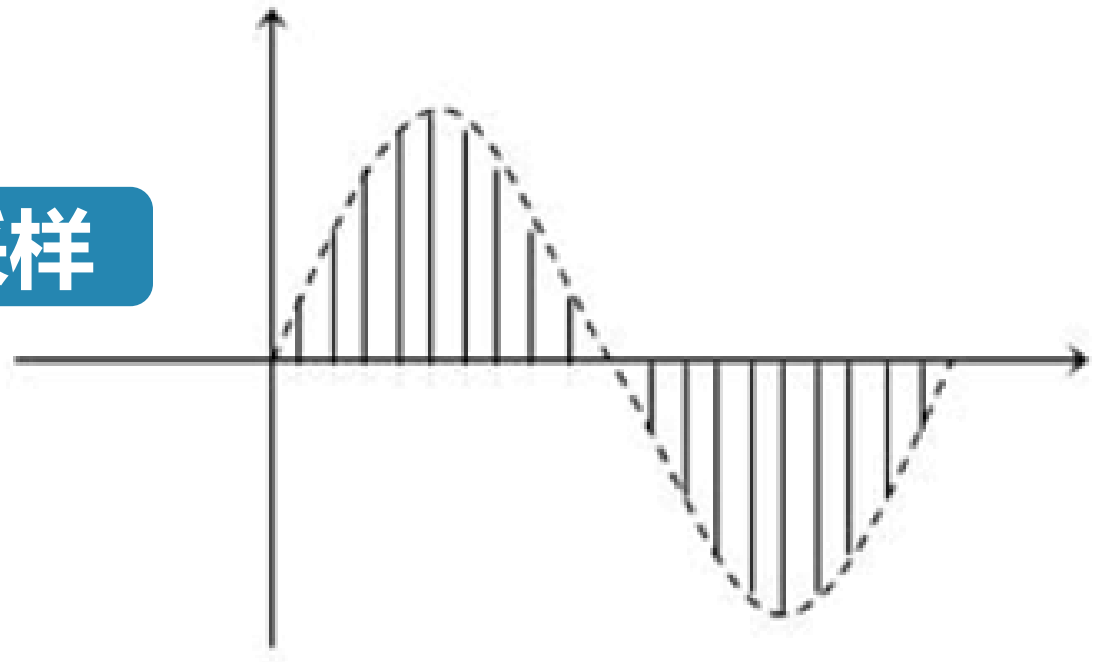
$$1024*768*24\text{位}/8\text{位}*30\text{帧/秒}*60\text{秒}=3.955\text{G}$$

思考：数据量如此巨大该如何解决

声音的数字化

采样是把时间上连续的模拟信号在时间轴上离散化的过程。

采样

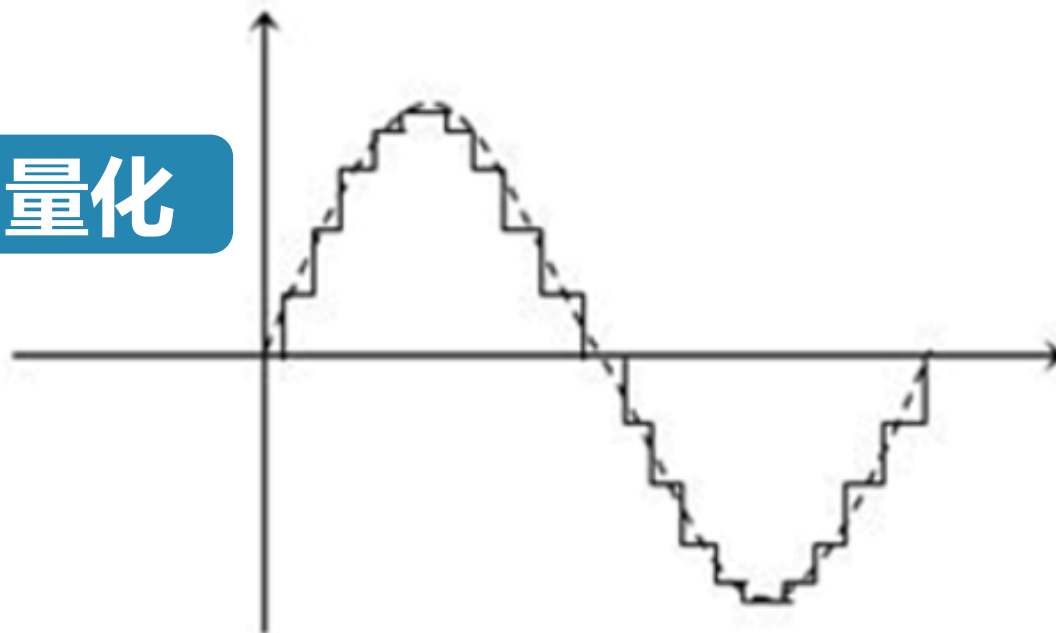


(b) 声音信号的采样

声音的数字化

量化的主要工作
就是将幅度上连续
取值的每一个样本
转换为离散值表示

量化



(c) 量化后的声音信号



声音的数字化

压缩编码

数据传输率 (bps) = 采样频率*精度*声道数

数据量 (Byte) = 数据传输率*持续时间/8



声音的数字化

压缩编码

CD唱片上所存储的立体声高保真音乐的采样频率为44.1kHz，量化精度为16位，双声道，计算一小时的数据量有多大？

$$44.1\text{kHz} \times 16\text{bit} \times 2 \times 3600\text{s} / 8 = 6350400\text{B} \approx 605.6\text{MB}$$



声音的数字化

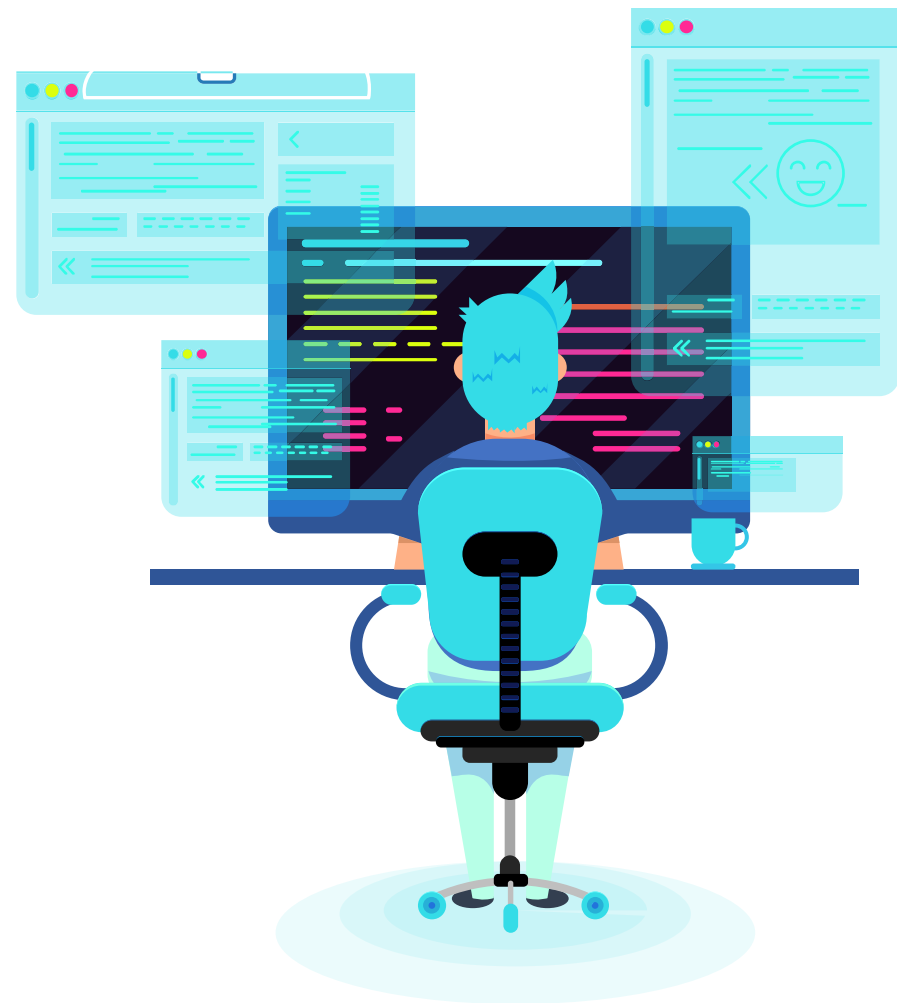
数字音频

- 电子合成的声音不需要从外界采样量化，因此数据量很小，它分为音乐合成和语音合成。例如MIDI (Musical Instrument Digital Interface, 乐器数字接口) 音频。
- 它描述一段音乐的音符、音调、使用什么乐器等，并通过音乐或声音合成器解释播放，产生音乐或语音



第五节：条形码

一维条形码
二维条形码





一维条形码

一维条形码(Bar Code)技术最早产生在二十世纪二十年代，诞生于威斯汀豪斯(Westinghouse)的实验室里，家喻户晓的箭牌口香糖是最早被打上条形码的。



一维条形码

条形码一共13位

前缀：1-3位，标识国家地区

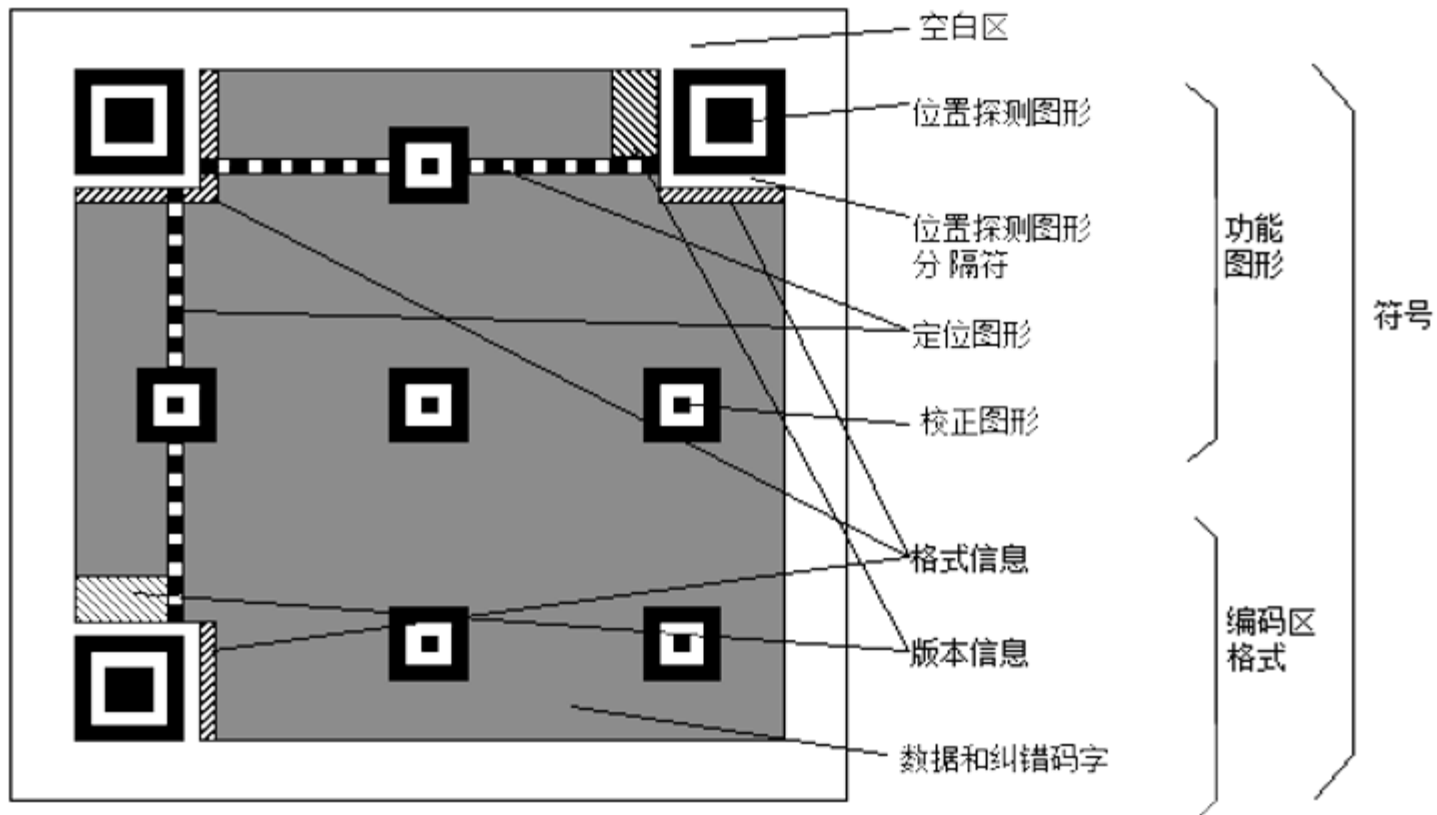
制造厂商代码：4-7位，标识生产厂家

商品代码：8-12，标识商品名称

校验码：第13位



二维条形码





二维条形码

- 以QR(Quick-Response) CODE为例，主要由位置探测图形、定位图形、格式信息、版本信息、数据和纠错信息等部分构成。



二维条形码

位置探测图形：

用于对二维码的定位，3个定位图形可标识一个矩形，同时可以用于确认二维码的大小和方向；

定位图形：

帮助定位的标线；

格式信息：

用于存放一些格式化数据。表示该二维码的纠错级别；



二维条形码

版本信息：

即二维码的规格，例如版本号7以上的二维码，需要预留两块3*6的区域存放一些版本信息。

数据信息和纠错信息：

实际保存的二维码信息(Data Code 数据码)和纠错信息（纠错码），用于修正二维码损坏带来的错误。



第二章（实践）：Windows操作系统和地质大数据库

- 第一节：Windows基本操作
- 第二节：Windows程序、文件和设备
- 第三节：地质大数据库查询
- 第四节：地质大数据库应用



Windows操作系统

01

Windows基本操作

02

Windows程序管理

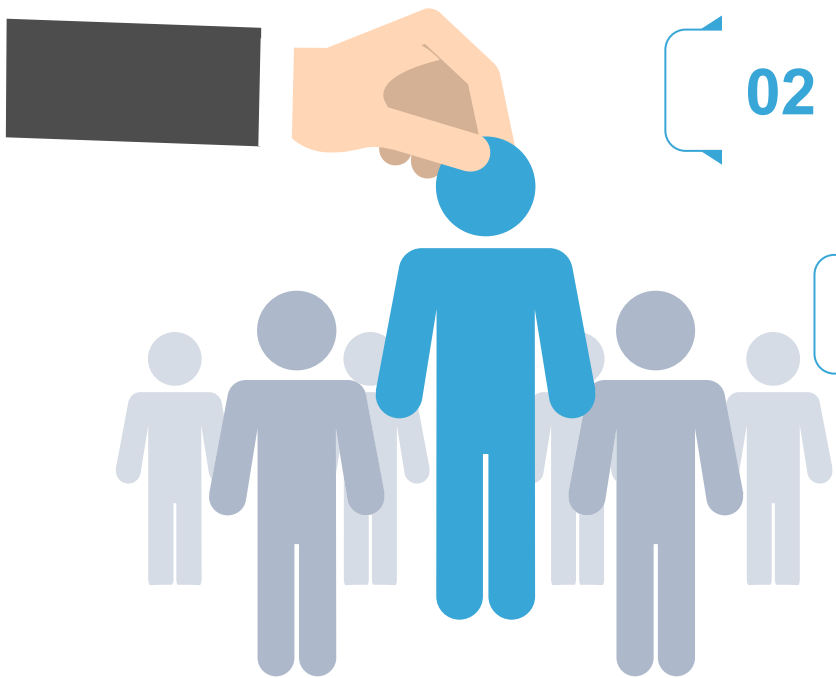
03

Windows文件管理

04

Windows设备管理

- 熟悉Windows
主要功能
- 高效利用计算机

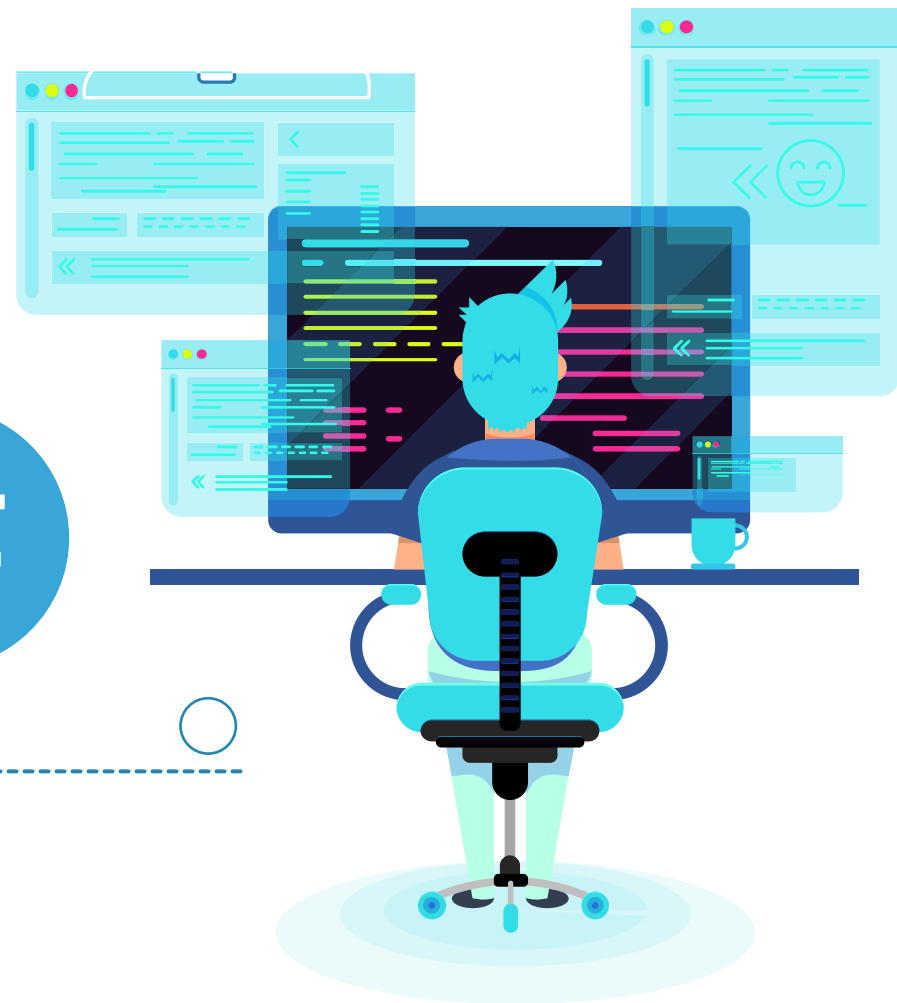
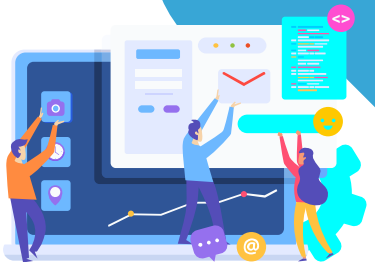




第一节：Windows基本操作



Windows基本操作





任务栏和桌面的设置

“win”键组合的使用

控制面板

用户账户

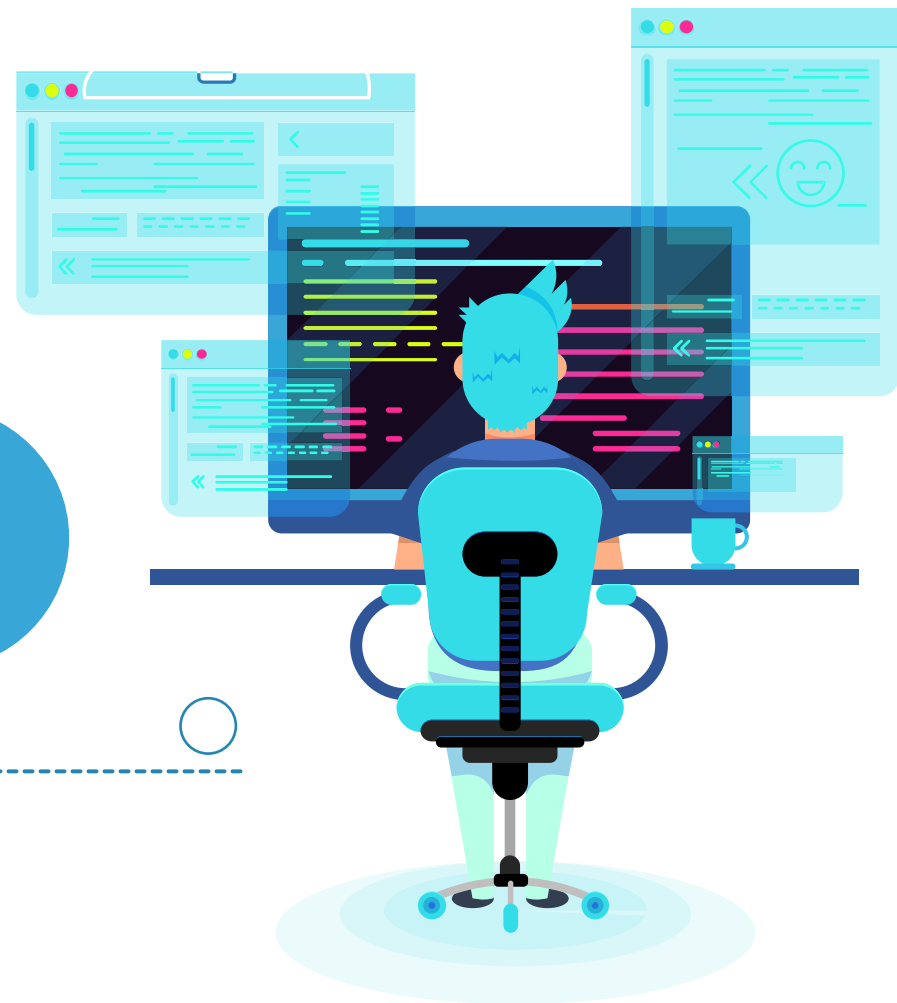
操作演示

快捷键：	作用
Win	打开/关闭开始菜单
Win + D	显示桌面（再按恢复）
Win + E	打开文件资源管理器
Win + L	锁定屏幕
Win + I	打开系统设置
Win + R	打开“运行”对话框
Win + S	打开搜索
Win + Tab	打开任务视图（多桌面）
Win + 方向键	窗口贴边/最大化/最小化
Win + Shift + S	截图（选区）



第二节：Windows程序文件

程序、文件和设备





01

启动程序

02

快捷方式

03

任务管
理器

04

查看组件
安装情况

05

卸载程序

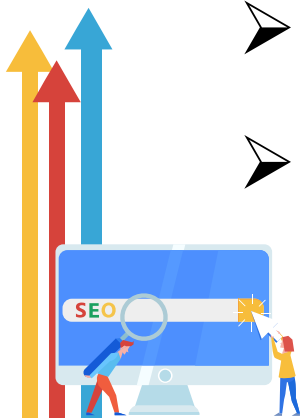
06

下载并安
装程序

Windows 程序管理

操作演示

- 文件资源管理器（C盘信息、查看方式、文件夹选项等）
- 文件和文件夹的操作（创建、删除、移动、复制、重命名等）
- 文件和文件夹的属性
- 搜索功能



Windows 文件管理

操作演示



01

计算机的系统信息

02

设备管理器

03

磁盘管理

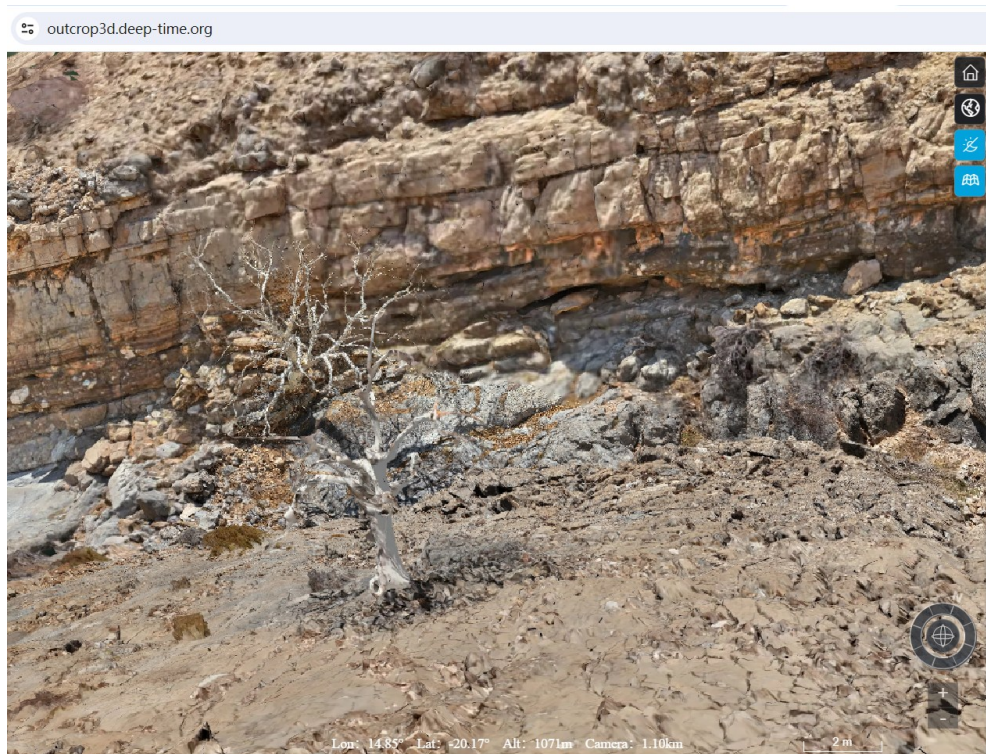


Windows 设备管理

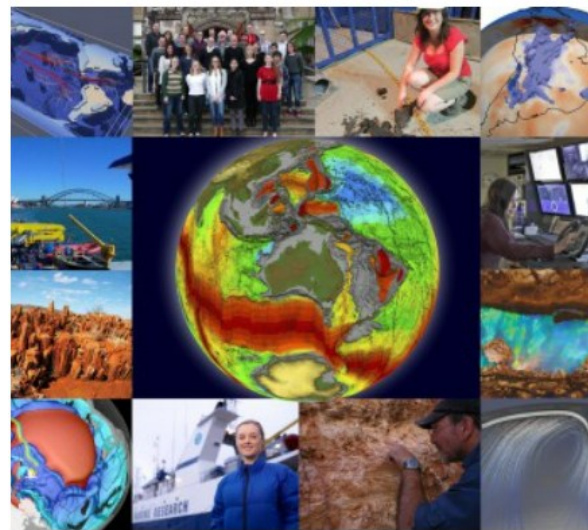
操作演示



第三节：地质大数据数据库查询



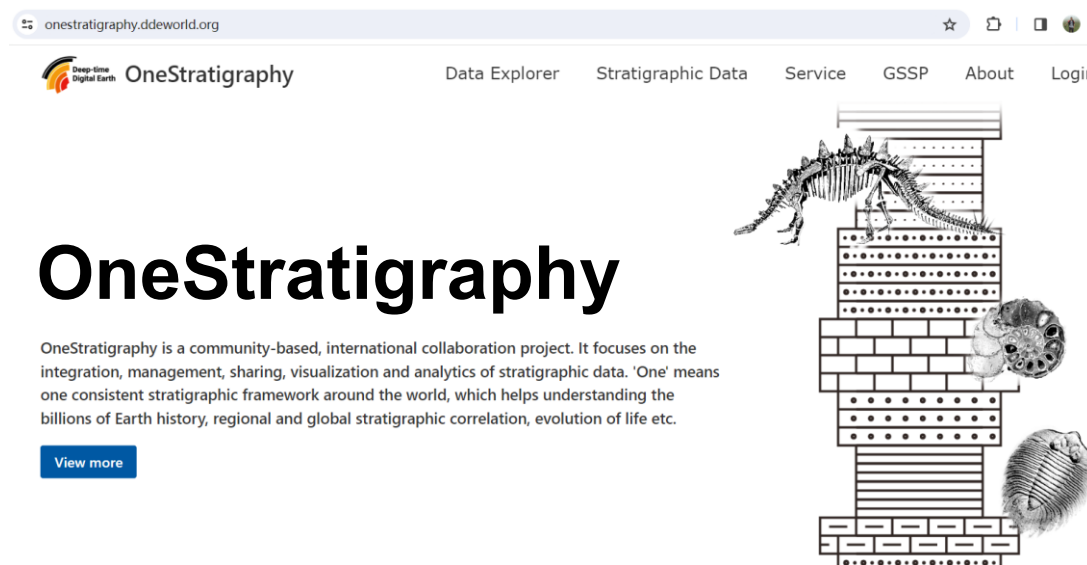
DDE-outcrop3D



What is EarthByte?

The EarthByte Research Group pursues geodata synthesis through space and time, building a deep-time virtual Earth. The group is well-known for open innovation via collaborative software development, "big data" analysis and its global open access digital data collections. The EarthByters lead the collaborative development of open-source virtual Earth software, including GPlates, pyGPlates, GPlately, the GPlates web service and portal, the GPlates App, pyBacktrack, and numerous machine learning workflows for spatial and spatio-temporal data and model analysis (see EarthByte GitHub repository). The GPlates ... [Read more...](#)

EarthByte





第四节：地质大数据数据库应用



earthchem.org/petdb?mapViewCenter=0,0



About Submit Data Access Data Communities Resources

PetDB 2.0 > Search

Add a filter to begin your search...

Filters

Analytical Features

Analyzed Materials

Chemical Variables

Laboratory

Sample Features

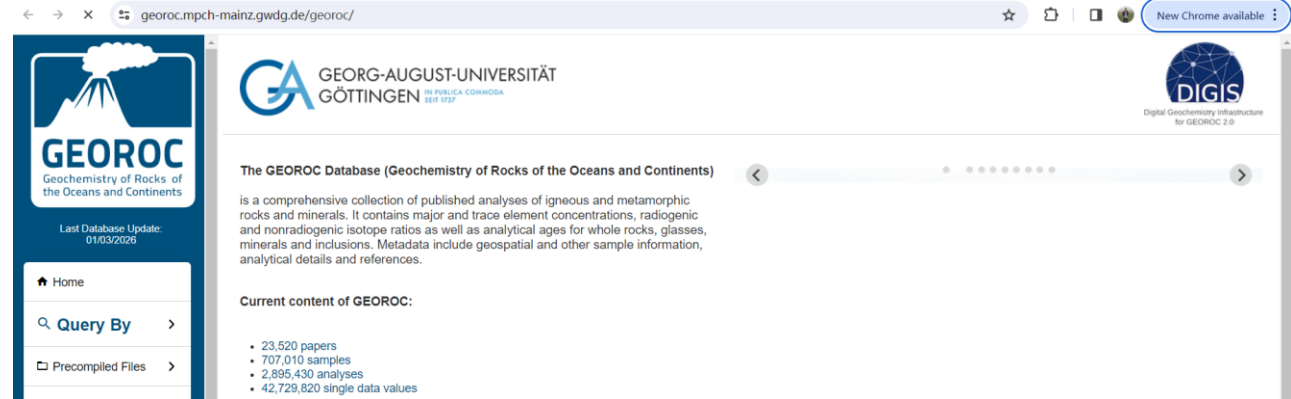
Additional Lookups

Welcome to PetDB 2.0

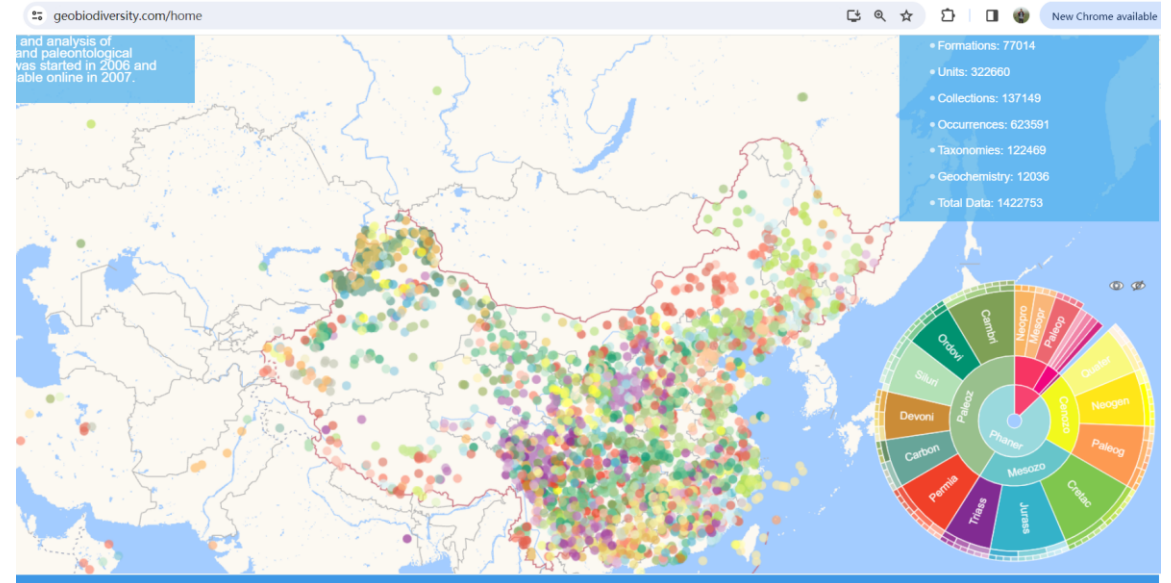
Search, extract, and integrate data
across publications.

7,374,200 Values 3,876 Citations

EARTHCHEM



GEOROC



Geobiodiversity



第二章：计算机中数据表示

- 计算机中的数制、数值、字符
- 计算机中的图像、音频和视频
- 条形码的原理及应用



第二章：Windows操作系统和地质大数据库

- Windows操作系统的基本操作
- Windows操作系统的程序、文件和设备管理
- 图形界面操作
- 命令行操作

- 地质大数据库查询
- 地质大数据库应用